

Università degli Studi dell'Insubria
Dipartimento di Scienze Teoriche e Applicate

CineMax

Manuale Tecnico

Autori

Donato Edoardo, 767352, VA

Gentile Giorgio, 759951, VA

Fezaj Cristina, 761382, VA

Versione documento: 1.0 | Data: 28/08/2026

Indice

1. Struttura dell'applicazione
2. Scelte architettureali
3. Scelte algoritmiche
4. Formato dei file e loro gestione
5. Strutture dati utilizzate
6. Gestione degli utenti
7. Gestione delle proiezioni
8. Gestione delle prenotazioni
9. Gestione degli errori e validazione degli input
10. Pattern e principi utilizzati
11. JavaDoc e commenti
12. Installazione ed esecuzione
13. Limiti della soluzione sviluppata
14. Sitografia / Bibliografia

1. Struttura dell'applicazione

Il progetto CineMax è stato sviluppato in Java ed è organizzato in diverse classi, ognuna con un compito specifico.

Le classi principali sono:

- **CineMax**: contiene il punto di ingresso del programma (main) e avvia l'applicazione.
- **MenuManager**: gestisce i menu e le principali operazioni disponibili per i diversi utenti.
- **Utente**: rappresenta un utente del sistema e contiene le sue informazioni personali, le credenziali e il ruolo.
- **Proiezione**: rappresenta una proiezione cinematografica, con informazioni sul film, sulla data e sull'orario.
- **Prenotazione**: rappresenta una prenotazione effettuata da un utente per una determinata proiezione.
- **FileManager**: si occupa del caricamento e del salvataggio dei dati nei file CSV.
- **Costanti**: contiene valori utilizzati in più parti del programma, come il separatore dei file CSV e il formato della data.

Le classi sono inserite nel package `cinemax`.

2. Scelte architetturali

Per organizzare il progetto abbiamo cercato di separare le responsabilità tra le diverse classi. MenuManager gestisce principalmente l'interazione con l'utente e le operazioni del sistema, mentre le classi Utente, Proiezione e Prenotazione rappresentano i dati principali dell'applicazione.

Per la gestione dei dati abbiamo utilizzato liste (ArrayList) e un FileManager separato, in modo da non mettere direttamente le operazioni sui file all'interno delle classi che rappresentano i dati.

Un'altra scelta importante è stata utilizzare degli ID numerici per identificare in modo univoco utenti, proiezioni e prenotazioni.

Il FileManager è stato progettato in forma generica, utilizzando i generics di Java (`<T>`) e le interfacce funzionali (`Function<>`). Invece di avere un metodo dedicato per ogni tipo di dato (ad esempio `caricaProiezioni`, `caricaUtenti`, `caricaPrenotazioni`), sono stati definiti due soli metodi, `carica()` e `salva()`, che funzionano con qualsiasi tipo di oggetto, a patto che venga fornita la funzione di conversione da e verso il formato CSV (ad esempio `Proiezione::fromCSV` e `Proiezione::toCSV`). Questa scelta evita di duplicare codice simile per ogni entità del progetto e rende il FileManager riutilizzabile anche per le entità aggiunte successivamente (Utente, Prenotazione).

La logica principale del progetto è stata quella di avere più indipendenza e astrazione possibile tra i ruoli e le classi scritte, così da non doversi preoccupare della struttura di ogni funzionalità usata.

3. Scelte algoritmiche

Come carattere separatore per i CSV è stato scelto di usare "\$", in quanto carattere poco comune, così da ridurre il rischio che, usando un carattere come la virgola o il punto e virgola, l'utente possa inserire informazioni che "romperebbero" il CSV. Facendo ciò abbiamo quindi evitato di introdurre logiche di splitting di record avanzate. La soluzione presenta comunque una percentuale di rischio, ma è considerata molto bassa.

La creazione del CSV e la sua deserializzazione sono definite da 3 metodi presenti in ogni classe considerata serializzabile:

- **fromCSV**: a partire da un record CSV esegue uno split e restituisce l'oggetto desiderato se il parsing va a buon fine
- **toCSV**: riordina i campi dell'oggetto separandoli tramite il carattere separatore
- **header**: restituisce la riga di intestazione per i CSV

Abbiamo inoltre introdotto una logica alquanto semplice di log system. Ciò ci ha permesso di poter riconoscere al volo eventuali errori progettuali, così da correggerli senza ridurre ulteriormente l'efficienza dello sviluppo.

Per la ricerca degli elementi all'interno delle liste vengono utilizzati principalmente gli Stream di Java, con operazioni come filter(), findFirst() e toList(). Ad esempio, per cercare una proiezione viene controllato l'ID:

```
return this.pProiezioni.stream()
    .filter(proiezione -> proiezione.getIdProiezione() == id)
    .findFirst()
    .orElse(null);
```

Per mostrare le proiezioni disponibili vengono invece filtrate quelle future e ordinate in base alla data e all'ora:

```
List<Proiezione> proiezioniFuture = this.pProiezioni.stream()
    .filter(it -> it.getDataOraProiezione().isAfter(LocalDateTime.now()))
    .sorted(Comparator.comparing(Proiezione::getDataOraProiezione))
    .toList();
```

Per controllare che due proiezioni non si sovrappongano viene calcolato l'orario di fine di ogni proiezione aggiungendo la durata del film all'orario di inizio. Se gli intervalli temporali si sovrappongono, la nuova proiezione non viene inserita.

Per la gestione dei posti viene calcolato il numero di posti già occupati sommando i posti delle prenotazioni associate alla stessa proiezione. La capienza massima del cinema è di 200 posti.

Per le password è stato utilizzato BCrypt, che è una libreria che si occupa di cifratura e decifratura in maniera molto efficace, in modo da non salvarle direttamente in chiaro nei file.

Qualsiasi dato viene inoltre ordinato per il proprio ID numerico, così da fornire comfort visivo all'utente finale durante la visualizzazione dei dati stessi.

4. Formato dei file e loro gestione

I dati del programma vengono salvati in file CSV nella cartella data.

La gestione dei file è affidata alla classe FileManager, mentre ogni classe che rappresenta un dato fornisce i metodi necessari per convertire i propri oggetti nel formato CSV. Come separatore dei campi è stato scelto il carattere \$, definito nella classe Costanti (come precedentemente anticipato):

```
public static final String SEPARATORE_CSV = "$";
```

La classe Proiezione, ad esempio, utilizza il metodo toCSV() per trasformare un oggetto in una riga del file:

```
return String.join(Costanti.SEPARATORE_CSV,
    String.valueOf(idProiezione),
    dataOraProiezione.toString(),
    titoloFilm,
    genere,
    regista,
    String.valueOf(anno),
    String.valueOf(durataMinuti),
    String.valueOf(etaMinima),
    String.valueOf(prezzoBiglietto));
```

Per effettuare l'operazione inversa viene utilizzato il metodo fromCSV(), che legge una riga e ricostruisce un oggetto Proiezione.

I file utilizzati dal programma sono principalmente:

- utenti.csv, per gli utenti;
- proiezioni.csv, per le proiezioni;
- prenotazioni.csv, per le prenotazioni.

Il FileManager viene inizializzato con il percorso:

```
this.pFileManager = new FileManager("data");
```

Il percorso di ogni file viene costruito unendo il nome della cartella dati (dataDirectory, es. "data") con il nome del file specifico (es. "proiezioni.csv"), tramite il metodo Path.of() di Java.

All'avvio del programma il metodo pCaricaDati() carica i dati presenti nei file e li inserisce nelle rispettive liste. Quando vengono effettuate modifiche, come una nuova prenotazione o una modifica di una proiezione, i dati vengono nuovamente salvati nei file.

5. Strutture dati utilizzate

Per rappresentare una singola proiezione abbiamo creato la classe Proiezione, che raggruppa tutti i suoi dati (data e ora, titolo, genere, regista, anno, durata, età minima, prezzo) come campi di tipo diverso (date, numeri, testo), invece di tenerli come semplice testo come sono scritti nel file CSV.

Per contenere più proiezioni insieme abbiamo usato una `List<Proiezione>`, una lista che può contenere un numero variabile di oggetti `Proiezione`.

A grandi linee lo stesso discorso vale anche per utenti e prenotazioni.

6. Gestione degli utenti

La classe Utente contiene i dati personali dell'utente, le credenziali e il ruolo. Sono previsti tre ruoli:

```
public enum Ruolo {  
    CLIENTE,  
    PROIEZIONISTA,  
    BIGLIETTAIO  
}
```

Il ruolo permette di mostrare menu differenti in base al tipo di utente che effettua il login. La classe contiene anche il metodo `verificaPassword()` e nel `MenuManager` viene utilizzato `BCrypt` per confrontare la password inserita con quella salvata.

7. Gestione delle proiezioni

La classe Proiezione contiene le informazioni relative alla proiezione, tra cui:

- ID della proiezione
- data e ora
- titolo del film
- genere
- regista
- anno
- durata
- età minima
- prezzo del biglietto

La classe contiene anche un costruttore di copia:

```
public Proiezione(Proiezione altraProiezione) {  
    this.idProiezione = altraProiezione.idProiezione;  
    this.dataOraProiezione = altraProiezione.dataOraProiezione;  
    this.titoloFilm = altraProiezione.titoloFilm;  
    this.genere = altraProiezione.genere;  
    this.regista = altraProiezione.regista;  
    this.anno = altraProiezione.anno;  
    this.durataMinuti = altraProiezione.durataMinuti;  
    this.etaMinima = altraProiezione.etaMinima;  
    this.prezzoBiglietto = altraProiezione.prezzoBiglietto;  
}
```

Questo costruttore viene utilizzato quando si vuole creare una copia della proiezione prima di applicare delle modifiche. L'ID viene mantenuto all'interno dell'oggetto tramite `idProiezione` e viene utilizzato per collegare una proiezione alle relative prenotazioni.

8. Gestione delle prenotazioni

Le prenotazioni collegano un utente a una proiezione e contengono il numero di posti prenotati.

Quando viene effettuata una prenotazione, il programma controlla:

- che la proiezione esista;
- che la proiezione sia futura;
- che l'età dell'utente sia sufficiente;
- che ci siano abbastanza posti disponibili.

Il numero di posti disponibili viene calcolato sottraendo alla capienza massima di 200 posti il numero di posti già prenotati per quella proiezione.

9. Gestione degli errori e validazione degli input

Per evitare inserimenti non validi sono stati creati alcuni metodi dedicati alla lettura degli input, come `pLeggiIntero()`, `pLeggiDouble()` e `pLeggiTesto()`. Questi metodi controllano che l'utente inserisca valori corretti e, quando necessario, che siano compresi in un determinato intervallo.

Per le date viene utilizzato `LocalDate` o `LocalDateTime` insieme a `DateTimeFormatter`. In caso di formato non valido viene gestita la `DateTimeParseException`.

Durante il caricamento dei file CSV vengono inoltre controllati il numero dei campi e la validità dei dati.

Questa soluzione consente di avere dei controlli consistenti e robusti senza ridondanza di codice o bug strutturali.

10. Pattern e principi utilizzati

Nel progetto è stato utilizzato in modo implicito un pattern simile allo Strategy pattern nella classe `FileManager`, dove il comportamento di conversione (da e verso CSV) viene passato come parametro sotto forma di funzione, invece di essere scritto fisso all'interno della classe. Oltre a questo, sono stati seguiti alcuni principi di organizzazione del codice.

11. JavaDoc e commenti

Il codice è stato documentato utilizzando la JavaDoc, per rendere esplicito lo scopo di classi, metodi e parametri anche a chi non ha sviluppato direttamente quella parte del codice. Ogni metodo pubblico riporta una descrizione del proprio funzionamento, l'elenco dei parametri (tag `@param`), il valore restituito (tag `@return`) e le eccezioni sollevate (tag `@throws`), quando presenti.

L'obiettivo è rendere il codice più comprensibile anche a una persona che non lo ha sviluppato direttamente.

12. Installazione ed esecuzione

Requisiti: Java Development Kit (JDK) versione 25 o superiore.

Il progetto utilizza la libreria esterna jbcrypt (jbcrypt-0.4.jar), inclusa nella cartella `lib/` del repository.

Compilazione (dalla cartella radice del progetto):

```
javac -cp lib/jbcrypt-0.4.jar -d bin src/cinemax/*.java
```

Esecuzione:

```
java -cp "bin;lib/jbcrypt-0.4.jar" cinemax.CineMax
```

Il progetto è organizzato secondo la seguente struttura di cartelle, come da specifiche di consegna:

- `src/`: codice sorgente
- `bin/`: file eseguibile (.jar)
- `data/`: file di dati (utenti.csv, proiezioni.csv, prenotazioni.csv)
- `lib/`: librerie esterne (jbcrypt)
- `doc/`: manuale utente, manuale tecnico, javadoc generata

13. Limiti della soluzione sviluppata

- Interfaccia a riga di comando (CLI), senza GUI.
- Gestione di una singola sala cinematografica con capienza fissa di 200 posti; non è prevista la gestione di più sale o cinema multipli.
- Persistenza dei dati su file CSV anziché su database relazionale: non è garantita la consistenza in caso di accessi concorrenti da più istanze del programma sugli stessi file.
- Separatore CSV personalizzato (§) invece di uno standard, per evitare conflitti con virgole nei campi testuali; non è tuttavia gestito l'escaping di eventuali caratteri § presenti nei dati stessi.
- Nessuna integrazione con sistemi di pagamento reali: il totale della prenotazione viene calcolato ma non è gestito un pagamento effettivo.

14. Sitografia / Bibliografia

- Oracle, *Java SE Documentation*, Online: <https://docs.oracle.com/en/java/javase/>
- Oracle, *How to Write Doc Comments for the Javadoc Tool*, Online: <https://www.oracle.com/technical-resources/articles/java/javadoc-tool.html>
- jBCrypt, libreria per l'hashing sicuro delle password, Online: <https://www.mindrot.org/projects/jBCrypt/>
- Oracle, *The Java Tutorials - Generics*, Online: <https://docs.oracle.com/javase/tutorial/java/generics/>
- Oracle, *java.util.function.Function - Java Platform SE Documentation*, Online: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/function/Function.html>